

Hadoop Practical No 2 – Word Count MapReduce

STEP 1 — Go to Home Directory

```
cd ~
```

STEP 2 — Create Project Folder

```
mkdir ~/wordcount
```

```
cd ~/wordcount
```

STEP 3 — Create Mapper File

```
nano WCMapper.java
```

WCMapper.java

```
import java.io.IOException;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapred.*;

public class WCMapper extends MapReduceBase
implements Mapper<LongWritable, Text, Text, IntWritable> {

    public void map(LongWritable key, Text value,
                    OutputCollector<Text, IntWritable> output,
                    Reporter rep) throws IOException {

        String line = value.toString();

        for(String word : line.split(" ")) {
            if(word.length() > 0) {
                output.collect(new Text(word),
                              new IntWritable(1));
            }
        }
    }
}
```

STEP 4 — Create Reducer File

```
nano WCReducer.java
```

```
import java.io.IOException;
import java.util.Iterator;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapred.*;

public class WCReducer extends MapReduceBase
implements Reducer<Text, IntWritable, Text, IntWritable> {

    public void reduce(Text key, Iterator<IntWritable> values,
                      OutputCollector<Text, IntWritable> output,
                      Reporter rep) throws IOException {

        int count = 0;

        while(values.hasNext()) {
            count += values.next().get();
        }

        output.collect(key, new IntWritable(count));
    }
}
```

STEP 5 — Create Driver File

```
nano WCDriver.java
```

```
import java.io.IOException;
import org.apache.hadoop.conf.Configured;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapred.*;
```

```

import org.apache.hadoop.util.*;

public class WCDriver extends Configured implements Tool {

    public int run(String args[]) throws IOException {

        JobConf conf = new JobConf(WCDriver.class);

        FileInputFormat.setInputPaths(conf, new Path(args[0]));
        FileOutputFormat.setOutputPath(conf, new Path(args[1]));

        conf.setMapperClass(WCMapper.class);
        conf.setReducerClass(WCReducer.class);

        conf.setMapOutputKeyClass(Text.class);
        conf.setMapOutputValueClass(IntWritable.class);

        conf.setOutputKeyClass(Text.class);
        conf.setOutputValueClass(IntWritable.class);

        JobClient.runJob(conf);

        return 0;
    }

    public static void main(String args[]) throws Exception {
        ToolRunner.run(new WCDriver(), args);
    }
}

```

STEP 6 — Compile Java Files

```
javac -classpath `hadoop classpath` -d . WCDriver.java WCReducer.java WCDriver.java
```

STEP 7 — Create JAR File

```
jar -cvf wordcount.jar *.class
```

STEP 8 — Create Input File

```
nano input.txt
```

Write:

```

hadoop is big data
hadoop is useful
big data is powerful

```

STEP 9 — Upload Input File to HDFS

```

hdfs dfs -mkdir /input
hdfs dfs -put input.txt /input
hdfs dfs -ls /input

```

STEP 10 — Run MapReduce Program

```
hadoop jar wordcount.jar WCDriver /input /output
```

STEP 11 — View Output

```
hdfs dfs -cat /output/part-00000
```

Expected Output:

```

big 2
data 2
hadoop 2
is 3
powerful 1
useful 1

```

STEP 12 — Before Running Again

```
hdfs dfs -rm -r /output
```

Important Viva Points

- Mapper converts words into (word,1)
- Reducer adds counts of same words
- Driver connects Mapper and Reducer